

SILVER → GOLD TRANSFORMATION PIPELINE

Customer Churn Analytics — Technical & Data Quality Report

Prepared for: Data Engineering & Analytics Stakeholders

Platform: CentOS 7 · Hadoop HDFS · Apache Spark 3.1.2 (PySpark) · Hive

Report Date: September 4, 2026

1. Executive Summary

This report documents a production-grade ETL pipeline that transforms cleaned Silver-layer customer data into a curated Gold-layer star schema for churn analytics and business intelligence. The pipeline is implemented in PySpark on a Hadoop/HDFS platform (Spark 3.1.2) and produces three Gold datasets — a customer dimension, a monthly customer-activity fact table, and a monthly churn-rate summary — which are subsequently exposed to BI tools such as Power BI through external Hive tables.

The pipeline combines four Silver sources (customers, support tickets, offers, and usage) into a conformed data model, enforces referential integrity so that only valid customers and their associated activity reach the Gold layer, and runs a 20-point automated data-quality check suite before certifying the output as PASS or CHECK. Built-in safety controls (SAFE_MODE) prevent accidental overwrite of existing Gold data, and a DEBUG flag provides schema, sample, and row-count visibility at every stage.

Overall, the design reflects sound data-warehousing practice: surrogate keys, slowly-changing-dimension-style start/end date columns, explicit null and duplicate handling, and a comprehensive reconciliation report. Section 7 highlights a small number of areas — primarily around key generation at scale and a few validation edge cases — worth reviewing before this pipeline is promoted to a high-volume production schedule.

2. Purpose & Scope

The pipeline's objective is to produce trusted, analytics-ready ('Gold') data for customer churn reporting. It consumes four Silver datasets that have already passed upstream cleaning and validation, and applies business rules, aggregation, and key generation to produce a dimensional model suitable for self-service BI and ad-hoc SQL analysis.

Item	Detail
Source layer	Silver (/data/silver/*) — cleaned, validated inputs
Target layer	Gold (/data/gold/*) — curated, reporting-ready outputs
Processing engine	PySpark 3.1.2 batch job
Storage format	Apache Parquet on HDFS
Consumption layer	Hive external tables → Power BI / analytics tools
Orchestration mode	Single script, idempotent per run (subject to SAFE_MODE)

3. Source Data (Silver Layer)

The pipeline reads four Silver datasets, each required to exist before processing begins; the job aborts with a clear error if any input path is missing.

Dataset	HDFS Path	Role in Pipeline
Customers	/data/silver/customers	Base population for the customer dimension (churn flag, product count)
Support Tickets	/data/silver/customer_support_tickets	Drives ticket-count and resolution-time metrics

Dataset	HDFS Path	Role in Pipeline
Offers	/data/silver/offers	Drives offer volume and acceptance-rate metrics
Usage	/data/silver/usage	Drives balance and usage-event metrics; also feeds lifetime value

4. Gold Data Model

The output is a classic star-schema pattern: one dimension table (dim_customer) and one fact table (fact_customer_activity), plus a pre-aggregated summary table (monthly_churn_rate) for fast top-level reporting.

4.1 dim_customer — one row per valid customer

Column	Description
cust_key	Surrogate key (row-number generated), unique, non-null
customer_id	Natural/business key from Silver
is_churned	0/1 churn flag
CLV_LTV	Customer lifetime value, summed from usage balances
avg_ticket_res_time_hrs	Average support ticket resolution time
num_products	Number of products held
dw_start_date	Warehouse validity start date
dw_end_date	Warehouse validity end date (open-ended: 9999-12-31)

4.2 fact_customer_activity — one row per customer + activity month

Column	Description
customer_id / cust_key	Business key and FK to dim_customer
activity_month	Period grain, format YYYY-MM
usage_events / total_balance / avg_balance	Usage metrics for the month
ticket_count / avg_resolution_time	Support metrics for the month
offer_count / accepted_offers / acceptance_rate	Offer metrics for the month
activity_key	Surrogate key (row-number generated), unique, non-null

4.3 monthly_churn_rate — one row per activity month

Column	Description
activity_month	Reporting period, format YYYY-MM
active_customers	Distinct customers with activity in the month

Column	Description
churned_customers	Distinct active customers flagged as churned
churn_rate	churned_customers ÷ active_customers, bounded 0–1

4.4 Processing Flow

Silver Customers → dim_customer. From dim_customer, the pipeline branches in two directions: (a) a validated customer-ID reference used to filter all downstream activity, and (b) monthly usage, ticket, and offer activity, which are unioned and joined to produce fact_customer_activity. fact_customer_activity is then aggregated by month to produce monthly_churn_rate.

5. Processing Logic (Step-by-Step)

The script executes as 21 sequential, logged steps. A summary of each stage follows.

1. Environment check — validates SAFE_MODE/DEBUG settings and confirms all four Silver paths exist on HDFS before any processing starts.
2. Read Silver — loads the four Parquet datasets and, in DEBUG mode, prints schema, a 10-row sample, and row counts for each.
3. Standardize customer keys — normalizes CustomerId/customerid/customer_id column-naming variants to a single customer_id (long) across all four datasets.
4. Prepare activity months — derives a YYYY-MM activity_month for usage (with fallback parsing for YYYY-MM, YYYY/MM, and MM/YYYY string formats), tickets (from created_at), and offers (from date_offered).
5. Customer aggregations — computes CLV_LTV (sum of monthly balances per customer) and avg_ticket_res_time_hrs (average resolution time per customer).
6. Build dim_customer — normalizes Exited → is_churned and NumOfProducts → num_products, deduplicates by customer_id, joins in the CLV and ticket aggregates (nulls coalesced to 0), stamps dw_start_date/dw_end_date, and assigns a row-number surrogate cust_key ordered by customer_id.
7. Prepare final customer reference — derives the authoritative list of valid customer_id values from dim_customer; this list is the single gate that all downstream fact data must pass through.
8. Monthly usage aggregation — usage_events, total_balance, avg_balance per customer/month.
9. Monthly ticket aggregation — ticket_count, avg_resolution_time per customer/month.
10. Monthly offer aggregation — offer_count, accepted_offers, acceptance_rate per customer/month.
11. Build activity months — unions the distinct customer/month combinations from usage, tickets, and offers, then inner-joins against the valid-customer list to drop any activity for customers not present in the final dimension.
12. Build fact_customer_activity — left-joins usage/ticket/offer monthly aggregates onto the activity-month spine, coalesces missing activity to zero, attaches cust_key via an inner join to dim_customer, and assigns a row-number surrogate activity_key ordered by customer_id and activity_month.
13. Build monthly_churn_rate — joins fact activity to the churn flag, counts distinct active and churned customers per month, and computes churn_rate with a guard against division by zero.
14. Prepare Gold outputs — for each of the three Gold paths, checks whether it already exists: under SAFE_MODE=1 the job aborts; under SAFE_MODE=0 the existing path is deleted.
15. Write Gold Parquet — writes all three tables (legacy datetime rebase mode set for Spark 3.1.2 date compatibility); monthly_churn_rate is coalesced to a single file.
16. Reload Gold — clears the Spark catalog cache and re-reads all three Gold tables from disk, so subsequent checks validate what was actually persisted, not what remained in memory.

17. Row counts — prints row counts for all three Gold tables.
18. Data quality checks — executes the full 20-point validation suite described in Section 6.
19. Quality report — prints a structured report grouped by dimension, fact, measures, rates, and business-consistency checks.
20. Gold samples — prints sample rows from all three Gold tables for visual spot-checking.
21. HDFS verification — confirms each Gold path exists on disk and lists its part files, then prints a final PASS/CHECK status and stops the Spark session.

6. Data Quality & Validation Framework

Before the run is certified, the pipeline evaluates 20+ automated checks grouped into five categories. Any failing check flips the final status from PASS to CHECK and causes the script to exit with a non-zero status code, making it safe to wire into a scheduler with failure alerting.

6.1 Dimension integrity

- customer_id and cust_key must be non-null
- cust_key must be unique (no duplicate surrogate keys)
- customer_id must be unique (one row per customer)
- is_churned must be exactly 0 or 1, never null
- CLV_LTV and avg_ticket_res_time_hrs must not be negative

6.2 Fact key integrity

- activity_key must be non-null and unique
- No duplicate customer_id + activity_month combinations
- customer_id and cust_key on fact rows must be non-null
- No orphan fact rows — every cust_key in the fact table must exist in dim_customer (left-anti join check)
- activity_month must be non-null and match the YYYY-MM pattern

6.3 Measure sanity

- usage_events, total_balance, ticket_count, offer_count, and accepted_offers must all be non-negative

6.4 Rate bounds

- acceptance_rate must fall between 0 and 1
- churn_rate must fall between 0 and 1

6.5 Business consistency

- accepted_offers can never exceed offer_count
- churned_customers can never exceed active_customers in a given month

7. Operational Controls & Observations

7.1 Safety and debug controls

- `SAFE_MODE` (default 1) — stops the job with an explicit error if a Gold output directory already exists, preventing accidental overwrite; set `SAFE_MODE=0` to intentionally replace existing Gold data.
- `DEBUG` (default 1) — prints schema, a 10-row sample, and a count for every intermediate DataFrame, which is very useful for development but should typically be disabled (`DEBUG=0`) in a high-volume production schedule to reduce job runtime and log volume.
- `DW_START_DATE` is configurable via environment variable, defaulting to the run date, supporting SCD-style dimension versioning if the pipeline is extended to type-2 history in future.

7.2 Points worth reviewing before scaling

- Surrogate keys (`cust_key`, `activity_key`) are generated with `Window.orderBy(...) + row_number()` over an unpartitioned window. This forces all rows through a single partition for key assignment, which is correct but will not scale well as data volume grows; a monotonically_increasing_id()-based or hash-based key strategy would parallelize better for larger datasets.
- `dim_customer` join keys are the single gate for fact-table validity. This is a strong design choice for referential integrity, but any customer excluded from `dim_customer` for any reason will silently drop all of their activity — worth confirming this is the desired behavior for edge cases such as late-arriving customers.
- The reload-after-write step (Section 5, step 16) means checks validate the persisted Parquet files rather than in-memory DataFrames — a good practice that catches serialization or type-coercion issues, though it does add a second full read pass over the data.
- `monthly_churn_rate` is coalesced to a single output file; this is appropriate for its small row count (one row per month) but is called out here since `coalesce(1)` can become a bottleneck if applied to a larger table.
- The final exit code (`sys.exit(1)` on any failed check) makes this pipeline schedule-friendly for tools like Airflow or Oozie, since a `CHECK` status will correctly fail the job rather than silently succeeding.

8. Consumption Layer — Hive & BI

The Gold Parquet outputs are registered as three external Hive tables (`dim_customer_gold`, `fact_customer_activity_gold`, `monthly_churn_rate_gold`), pointing at the same HDFS locations the Spark job writes to. This lets BI tools such as Power BI query the Gold layer directly via standard SQL without re-processing the data.

Hive Table	Grain	Primary Use
<code>dim_customer_gold</code>	One row per customer	Customer attributes, lifetime value, churn flag
<code>fact_customer_activity_gold</code>	One row per customer per month	Usage, support, and offer activity metrics
<code>monthly_churn_rate_gold</code>	One row per month	Aggregate churn KPI reporting

8.1 Analytics queries included

A companion SQL script provides thirteen business-question queries built on top of the Gold tables. These demonstrate the model's intended use and can serve directly as the basis for BI dashboard panels.

- Highest-value customers by `CLV_LTV`
- Churn rate by customer value segment (High / Medium / Low)
- Months with the highest churn rate
- Churn trend over time

- Customers generating the most support tickets
- Relationship between support resolution time and churn
- Overall offer acceptance-rate effectiveness
- Relationship between number of products and customer value
- Relationship between account balance and churn
- Relationship between engagement (usage volume) and churn
- Effect of product count on churn rate
- Retention-priority customer list (high value, low engagement, not yet churned)
- Composite churn-risk segmentation (Already Churned / High-Value High-Risk / Medium-Value High-Risk / Highly Engaged / Stable)

9. Conclusion

This pipeline delivers a well-structured, auditable path from Silver to Gold for customer churn analytics. Its strongest features are the strict referential-integrity gate on the customer dimension, the comprehensive 20-point automated data-quality suite, and the safety controls that prevent accidental data loss. The accompanying Hive layer and thirteen pre-built business queries make the Gold model immediately usable for churn reporting and customer segmentation.

The main opportunities for improvement relate to scalability of surrogate-key generation and confirming that customers excluded upstream are meant to silently drop all associated activity. Neither issue affects correctness at current data volumes, but both are worth addressing before this pipeline is scheduled against substantially larger datasets.